CIVORA

DOC-CIV-BE-2026 • REV 2.4

CENTRALIZED REPORTING HUB & CONTROL PLANE

BACKEND IMPLEMENTATION ROADMAP

Domain-Driven Python & Django Architecture, Multi-Repository Ecosystem, Database Modelling & Delivery Sequence

`Civora_Backend (Python / Django / DRF / Celery)`
`Civora-PE_and_Stakeholder_Frontend`
`Civora-Admin_Frontend`
`Civora_Mobile_App`



CIVORA
Intelligent Construction. Clearer Control.

architecture
2026

CIVORA

TECHNICAL ARCHITECTURE SPECIFICATION
CIVORA MULTI-REPO ECOSYSTEM

0.0 Table of Contents

1.0 Engineering approach 3
2.0 Phase 1 — Backend foundation and project setup 4
3.0 Phase 2 — Database architecture and data modelling 5
4.0 Phase 3 — Authentication and authorization (RBAC) 6
5.0 Phase 4 — Organization and project management 7
6.0 Phase 5 — Field submission and evidence management 8
7.0 Phase 6 — WhatsApp and external channel integrations 9
8.0 Phase 7 — Task management and field performance 10
9.0 Phase 8 — Report generation and document management 11
10.0 Phase 9 — AI processing and report assistance 12
11.0 Phase 10 — Review, approval, and governance 13
12.0 Phase 11 — Notifications and stakeholder access 14
13.0 Phase 12 — Testing, security, and production deployment 15
14.0 Recommended Backend File & Folder Structure (Civora_Backend) 16
15.0 Delivery Sequence and Milestone Priorities 21
16.0 First Coding-Agent Task (Task 001 — Architecture & DB Foundation) 22
17.0 Non-Negotiable Engineering Principles & Final Recommendation 23

CIVORA
CIVORA CONTROL PLANE · OFFICIAL ENGINEERING ARCHITECTURE ROADMAP

INSTITUTIONAL DOCUMENT CONTROL & REPOSITORY SCOPE

Doc Ref	Version	Date	Target Stack	Connected Repositories	Status
DOC-CIV-BE-2026	v2.4	30 Oct 2026	Python 3.12 / Django 5 / DRF / PostgreSQL / Celery	Backend, PE/Stakeholder Web, Admin Web, Mobile App	APPROVED FOR TASK 001

1.0 Engineering approach

Build Civora's backend in phases. Each phase should produce a working, testable capability before the next phase begins.

Recommended sequence:

1. Backend foundation and project setup
2. Database architecture and data modelling
3. Authentication and authorization
4. Organization and project management
5. Field submission and evidence management
6. WhatsApp and external channel integrations
7. Task management and field performance
8. Report generation and document management
9. AI processing and report assistance
10. Review, approval and governance
11. Notifications and stakeholder access
12. Testing, security and production deployment

Do not implement every dashboard endpoint or integrate AI and WhatsApp simultaneously. Establish security, data integrity, and the submission pipeline first.

REPOSITORY ECOSYSTEM & TECHNOLOGY STACK

Civora is structured as a multi-repository architecture where the backend serves as a standalone centralized API and processing hub for three client applications:

Repository	URL	Role
Civora Backend	https://github.com/Civora40/Civora_Backend	Centralized REST API, database, background workers, WhatsApp/external integrations, and AI processing hub
PE & Stakeholder Frontend	https://github.com/Civora40/Civora-PE_and_Stakeholder_Frontend	Web dashboard for Project Engineers (submissions, tasks, report preparation) and Read-Only Stakeholders (report viewing/downloads)
Admin Frontend	https://github.com/Civora40/Civora-Admin_Frontend	Platform & Organization Administration web portal (tenant management, RBAC, governance, system monitoring)
Mobile App	https://github.com/Civora40/Civora_Mobile_App	Field operations mobile client for on-site submissions, evidence capture, and task execution

Core backend stack

- **Language & Framework:** Python 3.12+, Django 5.x, Django REST Framework (DRF) or Django Ninja
- **Database & ORM:** PostgreSQL 16+, Django ORM & Django Migrations
- **Validation & Serialization:** DRF Serializers / Pydantic v2 schemas
- **Async & Background Jobs:** Celery + Redis (webhook ingestion, media downloading, AI processing, PDF rendering, notifications)
- **Authentication & CORS:** JWT (`djangorestframework-simplejwt`) + secure session/token auth and `django-cors-headers` configured across the 3 client apps
- **API Documentation:** OpenAPI 3.1 (`drf-spectacular`) for typed client contract generation across Mobile, PE/Stakeholder, and Admin frontends

> **Before implementation:** inspect the actual repository, package versions, existing services, and deployment architecture. Do not replace an existing backend framework or working configuration without documenting the technical reason and obtaining approval.

2.0 Phase 1 — Backend foundation and project setup

Objective: Establish a reliable standalone Python/Django backend structure before implementing business features.

IMPLEMENTATION TASKS

1. Inspect the existing repository and document its current architecture.
2. Initialize and standardize the Django multi-environment project structure (`config/`, `apps/`, `common/`, `infrastructure/`).
3. Configure Python tooling (`pyproject.toml`, `ruff`, `mypy` / `django-stubs`, `pytest-django`).
4. Configure environment variables and startup validation (`django-environ` / `pydantic-settings`).
5. Connect PostgreSQL and configure connection pooling.
6. Configure Django ORM settings and baseline migrations.
7. Establish centralized exception handling and custom DRF exception handler.
8. Define consistent API response envelopes, pagination, and error conventions.
9. Configure request/payload validation using DRF Serializers / Pydantic schemas.
10. Configure CORS, CSRF, and rate-limiting policies for the Mobile, PE/Stakeholder, and Admin clients.
11. Set up structured JSON application logging (`structlog` / Python `logging`).
12. Configure Celery + Redis for background task execution.
13. Configure development, staging, and production settings modules + Docker Compose.
14. Add automated testing (`pytest`) and GitHub Actions CI checks.

Deliverables: Running Django API service, verified PostgreSQL and Redis connections, working migrations, environment validation, centralized error handling, OpenAPI schema endpoint, and initial `pytest` setup.

ACCEPTANCE CRITERIA

- The application starts reliably and connects to PostgreSQL.
- Migrations (`python manage.py migrate`) apply cleanly to a fresh development database.
- Invalid or missing environment configuration fails safely on startup with a clear error.
- Health check endpoint (`GET /api/v1/health/`) returns database and cache status.
- Initial automated test suite (`pytest`) passes.

3.0 Phase 2 — Database architecture and data modelling

Objective: Model Civora's business workflows and governance requirements before building dependent modules.

INITIAL DOMAIN ENTITIES

Domain	Suggested entities
Identity	User, Account, Session
Organization	Organization, OrganizationMembership
Access control	Role, Permission, RolePermission
Projects	Project, ProjectMembership, ProjectDepartment
Field operations	Submission, SubmissionAttachment
Tasks	Task, TaskAssignment
Reporting	Report, ReportVersion, ReportReview
Governance	AuditLog, Approval
Notifications	Notification, NotificationPreference

These are starting points, not a mandate to create every table immediately. Refine the model against confirmed requirements and avoid premature abstraction.

IMPLEMENTATION TASKS

1. Define the organization and multi-tenancy model.
2. Define users and organization memberships.
3. Define project ownership and project memberships.
4. Define role and permission relationships.
5. Define project departments and team assignments.
6. Define submission and attachment relationships.
7. Define task and assignment relationships.
8. Define report, review, and approval relationships.
9. Define audit and notification records.
10. Create migrations and development seed data.
11. Document key relationships, constraints, and deletion policies.

MULTI-TENANCY REQUIREMENTS

- Every tenant-owned record must have a clear organization boundary.
- Project-owned records must be scoped to the relevant project.
- Enforce tenant and project boundaries in server-side queries and authorization checks.
- Do not rely on frontend visibility or client-supplied organization IDs as authorization.
- Use database constraints and indexes to support integrity and common access patterns.
- Define retention, archival, and deletion rules before implementing destructive operations.

Deliverables: Reviewed entity relationship model, Django ORM models across domain apps, initial migrations, development seed management command (`python manage.py seed_dev`), and data ownership/retention notes.

ACCEPTANCE CRITERIA

Core relationships have explicit foreign keys; tenant-owned records can be scoped reliably; unauthorized cross-tenant/project access is prevented; migrations run on a clean database; schema decisions are documented.

4.0 Phase 3 — Authentication and authorization (RBAC)

Objective: Ensure every user can access only the resources and actions they are authorized to use.

IMPLEMENTATION TASKS

1. Registration and account creation.
2. Login and logout.
3. Secure session management.
4. Email verification.
5. Password reset.
6. User profile management.
7. Organization membership management.
8. Role assignment.
9. Permission evaluation.
10. Project-level access checks.
11. API authorization middleware or service-level guards.
12. Audit logging for sensitive access and permission changes.

INITIAL ROLE ASSUMPTIONS

Confirm the canonical role list with the product owner and existing application before implementation. Roles discussed so far include:

- `platform_admin` — platform-level administration, subject to explicit scope.
- `project_engineer` — authorized project operations, submissions, tasks, and report preparation.
- `read_only_stakeholder` — access to explicitly authorized project information and released reports.

Do not assume a stakeholder can view every report or that a project engineer can approve their own final reports. Model permissions as explicit capabilities and enforce them server-side.

Deliverables: Authentication endpoints and session handling, organization membership and role assignment, permission evaluation service, project-level authorization guards, and security tests.

ACCEPTANCE CRITERIA

Unauthenticated requests cannot access protected resources; users cannot access another tenant's data; project membership and permissions are checked server-side; unauthorized actions return consistent errors; sensitive permission changes are audited.

5.0 Phase 4 — Organization and project management

Objective: Build the foundation for all project-related operations.

IMPLEMENTATION TASKS

1. Create organization endpoints.
2. Implement organization profile and settings.
3. Implement user invitations and membership management.
4. Build project creation.
5. Build project update and configuration.
6. Implement project status and lifecycle management.
7. Add project departments and work categories.
8. Assign project engineers and stakeholders.
9. Implement project membership and access validation.
10. Add project activity history.

SUGGESTED PROJECT LIFECYCLE

`Draft → Setup → Active → On Hold → Completed → Archived`

Allowed transitions must be defined as business rules and guarded by permissions. Do not allow arbitrary status changes from the client.

INITIAL ENDPOINT OUTLINE

```
POST   /api/organizations
GET    /api/organizations/:organizationId

POST   /api/projects
GET    /api/projects
GET    /api/projects/:projectId
PATCH  /api/projects/:projectId

POST   /api/projects/:projectId/members
GET    /api/projects/:projectId/members
PATCH  /api/projects/:projectId/members/:memberId
DELETE /api/projects/:projectId/members/:memberId
```

These routes are illustrative. Align naming, versioning, and route conventions with the existing API.

Deliverables: Organization management, project CRUD and lifecycle rules, project team/department management, and project activity history.

ACCEPTANCE CRITERIA

Authorized users can create and configure projects, assign members, and manage lifecycle; unauthorized users cannot view or modify project records; invalid lifecycle transitions are rejected.

6.0 Phase 5 — Field submission and evidence management

> **Objective:** Build the central system that receives, preserves, organizes, and processes field reports.

This is one of Civora's highest-priority modules because the reporting system depends on it.

IMPLEMENTATION TASKS

1. Create the submission data model.
2. Implement manual submission through the API.
3. Implement submission categories and departments.
4. Implement sender identity and attribution.
5. Record original submission time and server-received time separately.
6. Implement attachment uploads.
7. Store images, documents, and other evidence in object storage.
8. Link attachments to their original submissions.
9. Implement submission status tracking.
10. Build project-based and department-based filtering.
11. Implement search and pagination.
12. Add immutable submission history and audit events.

SUBMISSION RECORD REQUIREMENTS

Preserve, where available:

- Original message or report content
- Sender identity and assigned role
- Organization and project association
- Department or professional category
- Original submission timestamp
- Server-received timestamp
- Source channel
- Original attachment references
- Processing and verification status
- External provider message or event identifier, where applicable

Do not invent missing sender, timestamp, project, or department information. Represent missing values explicitly and route them for verification.

SUGGESTED SUBMISSION LIFECYCLE

`Received → Processing → Pending Verification → Verified → Included in Report`

Additional outcomes may include `Flagged`, `Rejected`, or `Clarification Required`. Define valid transitions, responsible roles, and transition history.

EVIDENCE INTEGRITY REQUIREMENTS

- Preserve original submissions and source payloads.
- Never overwrite original field evidence with edited or AI-generated content.
- Store annotations, corrections, and generated drafts separately.
- Record file metadata, ownership, storage key, checksum where appropriate, and upload status.
- Use private object storage and short-lived authorized download URLs.
- Validate file size, extension, and actual content type.
- Consider malware scanning for uploaded files.

> **Deliverables:** Submission API and service, evidence upload/retrieval service, search and filtering, status workflow, immutable source record, and audit trail.

> **ACCEPTANCE CRITERIA**
> A submission can be created, attributed, stored, and retrieved; original content remains unchanged; attachments are linked correctly; unauthorized users cannot retrieve evidence; missing information remains pending; duplicate requests do not create duplicate records where idempotency is required.

7.0 Phase 6 — WhatsApp and external channel integrations

Objective: Receive field submissions from communication channels used by project teams.

IMPLEMENTATION TASKS

1. Establish a provider-independent integration architecture.
2. Configure WhatsApp Business Platform credentials.
3. Implement webhook verification.
4. Receive and validate webhook events.
5. Store raw webhook payloads securely.
6. Implement idempotency to prevent duplicate submissions.
7. Map incoming phone numbers to verified Civora users or field identities.
8. Resolve project and department associations.
9. Retrieve and store supported media attachments.
10. Convert incoming messages into Civora submission records.
11. Implement retry handling and failed-event monitoring.
12. Add an integration activity log.

CRITICAL PROVIDER CAPABILITY CHECK

Validate actual supported group-ingestion capability before committing to the proposed nine-group workflow. Access to the WhatsApp Business Platform does not automatically mean the standard Cloud API can ingest arbitrary existing group conversations.

Run a technical proof of concept first. Document supported message types, group versus individual conversation support, webhook coverage, media retrieval limitations, sender identity information, rate limits, retention considerations, and provider terms.

Keep the integration layer provider-independent so Telegram or another approved source can be added later.

Deliverables: Webhook endpoint and verification, raw event storage, idempotent processing, sender mapping, project resolution, supported media ingestion, and integration monitoring.

ACCEPTANCE CRITERIA

Supported events are validated; replayed events do not create duplicate submissions; original provider payloads are preserved; unmapped senders or unresolved projects are flagged; failures can be retried safely; implementation reflects verified provider capabilities.

8.0 Phase 7 — Task management and field performance

Objective: Turn submitted information into trackable project activities.

IMPLEMENTATION TASKS

1. Create task and task-assignment models.
2. Implement task creation.
3. Assign tasks to project personnel.
4. Implement due dates and priorities.
5. Add task status transitions.
6. Link tasks to submissions and project activities.
7. Record task completion evidence.
8. Implement overdue-task detection.
9. Build activity and compliance metrics.
10. Generate weekly and monthly performance summaries.

PERFORMANCE POLICY REQUIREMENTS

The proposed 75–80% compliance threshold must be configurable, not hardcoded.

Before operational use, confirm the scoring formula, eligible activities and denominator, reporting period, exemptions, treatment of late/incomplete submissions, override authority, score visibility, and whether scores are informational or used in formal decisions.

Do not automatically trigger salary deductions or payroll decisions. Those require separate explicit authorization and approved policy.

Deliverables: Task and assignment APIs, task status/evidence workflow, activity metrics, configurable performance calculation, and weekly/monthly summaries.

ACCEPTANCE CRITERIA

Tasks can be assigned and tracked; completion evidence is retained; metrics are reproducible from underlying records; scoring rules are configurable and documented; overrides and exceptions are auditable.

9.0 Phase 8 — Report generation and document management

Objective: Transform project evidence into structured, reviewable reports.

IMPLEMENTATION TASKS

1. Define report templates.
2. Create the report data model.
3. Link reports to projects and reporting periods.
4. Select submissions and evidence for inclusion.
5. Implement report drafting.
6. Add structured report sections.
7. Implement report editing and saving.
8. Add report version history.
9. Implement tables, figures, and attachment references.
10. Build document rendering and PDF export.
11. Add DOCX export if required.
12. Implement branded covers, automatic table of contents, and pagination.
13. Add report preview and document download.

TRACEABILITY REQUIREMENTS

- Reports must retain links to source submissions and evidence.
- Record which evidence was included in each report version.
- Preserve prior report versions.
- Distinguish source facts, user-authored narrative, and AI-generated draft content.
- Do not silently change a released report.
- Any amendment to a released report must create a new version and follow the appropriate approval process.

Use one consistent rendering pipeline for preview and export. Avoid separate rendering logic that produces inconsistent page layouts.

Deliverables: Report and version APIs, evidence selection and traceability, structured report editor data model, document preview and PDF export, and optional DOCX export if in scope.

ACCEPTANCE CRITERIA

Reports can be created from authorized project evidence; included sources are traceable; editing creates controlled versions; preview and exports are consistent; released versions cannot be silently modified.

10.0 Phase 9 — AI processing and report assistance

Objective: Reduce manual work while preserving evidence integrity and human accountability.

IMPLEMENTATION TASKS

1. Establish an AI provider abstraction.
2. Configure model credentials securely.
3. Create structured prompts for submission classification.
4. Implement message summarization.
5. Identify missing information.
6. Detect potential duplicates and inconsistencies.
7. Generate report-section drafts from selected evidence.
8. Attach source references to generated statements where possible.
9. Store AI outputs separately from original records.
10. Implement human review and correction.
11. Track model, prompt version, and generation metadata.
12. Add usage limits, retries, and failure monitoring.

APPROPRIATE AI ASSISTANCE

AI may assist with submission classification, summarization, missing-information flags, potential duplicate detection, inconsistency identification, unusual activity flags, and draft report sections.

AI GOVERNANCE REQUIREMENTS

- AI must not invent missing facts.
- AI output must be distinguishable from original evidence.
- Generated statements should retain source references where possible.
- Users must be able to review and correct generated content.
- AI must not independently verify real-world events.
- AI must not approve or release reports.
- Log the model and prompt version used for material outputs.
- Treat model output as untrusted input and validate it before persistence or downstream use.

Deliverables: AI provider integration, structured processing services, evidence-linked draft outputs, human review workflow, and usage/failure monitoring.

ACCEPTANCE CRITERIA

AI outputs are stored separately from original evidence; generated content can be traced to input sources; missing information is flagged rather than fabricated; humans control acceptance and editing; AI cannot bypass approval or release permissions.

11.0 Phase 10 — Review, approval, and governance

Objective: Establish formal controls over report review and release.

IMPLEMENTATION TASKS

1. Create report review workflows.
2. Assign reviewers.
3. Implement review comments.
4. Implement revision requests.
5. Record reviewer decisions.
6. Implement approval permissions.
7. Separate submission verification from report approval.
8. Implement final report release authorization.
9. Lock released report versions against silent modification.
10. Maintain a complete approval audit trail.

SUGGESTED REPORT LIFECYCLE

```
Draft → Submitted for Review → Under Review → Revision Requested → Approved → Released
```

Define valid transitions, required reviewers, separation-of-duties rules, and who can release reports.

Every decision should record actor, action, timestamp, report and version, decision/status, and comments or reason where applicable.

Submission verification and final report approval are distinct controls.

Deliverables: Review and approval APIs, review assignment/comments, revision workflow, final release authorization, and approval audit history.

ACCEPTANCE CRITERIA

Only authorized users can review, approve, or release; the reviewed version is unambiguous; revision requests preserve previous versions; decisions are auditable; released reports cannot be silently overwritten.

12.0 Phase 11 — Notifications and stakeholder access

Objective: Deliver relevant project updates to the right people.

IMPLEMENTATION TASKS

1. Create the notification data model.
2. Implement in-app notifications.
3. Configure email notifications.
4. Add notification preferences.
5. Implement event-based notifications.
6. Add project-specific notification rules.
7. Build stakeholder report access.
8. Implement released-report downloads.
9. Add milestone and project update notifications.
10. Track notification delivery status.

EXAMPLE EVENT ROUTING

Event	Typical recipient
New field submission	Project Engineer
Submission requires clarification	Assigned submitter
Report submitted for review	Assigned reviewer
Revision requested	Report author
Report approved	Authorized stakeholders
Report released	Authorized project stakeholders

Recipient rules must be configurable and permission-aware. Stakeholders should only see information authorized for their project membership and access level.

Deliverables: Notification service, in-app and email delivery, preferences, event-to-recipient rules, and stakeholder report access.

ACCEPTANCE CRITERIA

Notifications originate from defined events; recipients are resolved using current permissions and project membership; failures are recorded and retryable; stakeholders cannot access unreleased or unauthorized reports.

13.0 Phase 12 — Testing, security, and production deployment

Objective: Make the backend production-ready and operationally supportable.

IMPLEMENTATION TASKS

1. Write unit tests for business logic.
2. Write integration tests for database operations.
3. Test authentication and RBAC.
4. Test cross-organization data isolation.
5. Test project-level authorization.
6. Test webhook processing and duplicate-event handling.
7. Test file upload security.
8. Test report versioning and approval workflows.
9. Test failure recovery and background jobs.
10. Conduct API security reviews.
11. Configure database backups.
12. Configure monitoring and error reporting.
13. Configure production deployment.
14. Run end-to-end acceptance tests.
15. Test backup restoration and operational recovery.

SECURITY CHECKLIST

- Rate limiting and abuse protection
- Secure session and cookie configuration
- Input validation and output encoding
- Secrets management
- Tenant and project authorization
- File type, size, and content validation
- Private object storage and authorized download URLs
- Malware scanning where appropriate
- Audit logging for sensitive operations
- Database backup and restore tests
- Dependency and vulnerability scanning
- Safe error responses that do not expose secrets or internal stack traces

Deliverables: Automated test suite, security review and remediation, monitoring and alerting, backup/restore procedure, production deployment documentation, and end-to-end acceptance report.

ACCEPTANCE CRITERIA

Critical authorization and data-isolation tests pass; core workflows pass end-to-end tests; production secrets are not committed; backups can be restored and verified; errors and integration failures are observable; deployment and rollback procedures are documented.

14.0 Recommended Backend File & Folder Structure (Civora_Backend)

Because `Civora_Backend` is a standalone repository serving three separate frontend repositories (`Civora_Mobile_App`, `Civora-PE_and_Stakeholder_Frontend`, and `Civora-Admin_Frontend`), organize the codebase using a **Domain-Driven Django Architecture**. Keep HTTP views/controllers thin, isolate business logic inside `services.py` and `selectors.py`, and decouple external providers (WhatsApp, S3/GCS, LLMs, Email) inside `infrastructure/`.

14.1 COMPLETE REPOSITORY DIRECTORY TREE

```
Civora_Backend/
├── .github/
│   └── workflows/
│       ├── ci.yml                  # Lint (ruff), typecheck (mypy), test (pytest)
│       └── deploy.yml              # Staging / Production deployment pipeline
│
├── config/                         # Django project configuration & root routing
│   ├── __init__.py
│   ├── celery.py                   # Celery app initialization & queue routing
│   ├── urls.py                     # Root URL dispatcher (/api/v1/, /webhooks/, /docs/)
│   ├── wsgi.py                     # WSGI entry point (Gunicorn/uWSGI)
│   ├── asgi.py                     # ASGI entry point (Uvicorn/Daphne)
│   └── settings/
│       ├── __init__.py
│       ├── env.py                  # Environment variable loading & strict validation
│       ├── base.py                 # Shared core settings (installed apps, middleware, ORM)
│       ├── development.py          # Local dev settings (debug, local storage, CORS)
│       ├── staging.py              # Staging environment overrides
│       ├── production.py           # Hardened production settings (SSL, security headers)
│       └── test.py                 # Fast test runner settings
│
├── apps/                           # Domain-driven Django applications
│   ├── __init__.py
│   │
│   ├── authentication/             # Phase 3: Auth, JWT/session management, password reset
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py               # Session/Token audit & verification tokens
│   │   ├── serializers.py          # Login, register, token refresh, password reset schemas
│   │   ├── services.py             # Auth workflows, email verification, token revocation
│   │   ├── views.py                # Auth endpoints (/api/v1/auth/...)
│   │   ├── urls.py
│   │   └── tests/
│   │
│   ├── users/                      # Phase 2 & 3: User identities & profiles
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py               # Custom User model, phone/WhatsApp identity mapping
│   │   ├── serializers.py
│   │   ├── selectors.py            # Read queries for user profiles
│   │   ├── services.py             # Profile updates, identity verification
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── organizations/              # Phase 2 & 4: Multi-tenant organizations & memberships
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py               # Organization, OrganizationMembership, Invitation
│   │   ├── serializers.py
│   │   ├── selectors.py
│   │   ├── services.py             # Tenant provisioning, member invitations
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── access_control/             # Phase 3: RBAC (Roles, Permissions, Guards)
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py               # Role, Permission, RolePermission
│   │   ├── permissions.py          # DRF permission classes (PlatformAdmin, ProjectEngineer, Stakeholder)
│   │   ├── selectors.py            # Permission evaluation queries
│   │   ├── services.py             # Role assignment & capability checks
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── projects/                   # Phase 4: Projects, departments, lifecycle & members
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py               # Project, ProjectDepartment, ProjectMembership, ProjectActivity
│   │   ├── serializers.py
│   │   ├── selectors.py            # Tenant- & membership-scoped project queries
│   │   ├── services.py             # Project CRUD, lifecycle state machine (Draft -> Active -> ...)
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
```

14.1 Backend Directory Tree — Submissions, Evidence, Integrations & Tasks (Cont.)

14.1 COMPLETE REPOSITORY DIRECTORY TREE — CORE OPERATIONS APPS (CONT.)

```
Civora_Backend/
├── apps/ (continued)
│   ├── submissions/                    # Phase 5: Central field submission engine
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                   # Submission (immutable source), SubmissionStatusHistory
│   │   ├── serializers.py
│   │   ├── filters.py                  # Project, department, status, date & search filtering
│   │   ├── selectors.py
│   │   ├── services.py                 # Submission ingestion, attribution, status transitions
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── evidence/                       # Phase 5: Attachments, media metadata & integrity
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                   # SubmissionAttachment, checksums, storage keys
│   │   ├── serializers.py
│   │   ├── validators.py               # MIME type, file size, extension & malware checks
│   │   ├── services.py                 # Presigned upload/download URLs, linking to submissions
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── integrations/                   # Phase 6: WhatsApp & external channel ingestion
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                   # WebhookEvent (raw payload), ChannelIdentityMapping, IntegrationLog
│   │   ├── serializers.py
│   │   ├── services.py                 # Idempotent event processing, sender/project resolution
│   │   ├── tasks.py                    # Celery tasks for async webhook processing & media fetch
│   │   ├── webhooks.py                 # Webhook verification & ingestion endpoints
│   │   ├── adapters/                   # Provider-independent channel adapters
│   │   │   ├── __init__.py
│   │   │   ├── base.py
│   │   │   └── whatsapp.py             # WhatsApp Business Cloud API adapter
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── tasks/                          # Phase 7: Task management & assignments
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                   # Task, TaskAssignment, TaskCompletionEvidence
│   │   ├── serializers.py
│   │   ├── selectors.py
│   │   ├── services.py                 # Task creation, assignment, status transitions
│   │   ├── tasks.py                    # Scheduled Celery job for overdue-task detection
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── performance/                    # Phase 7: Field compliance & performance metrics
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                   # PerformancePolicy, PerformanceSummary, ScoreOverride
│   │   ├── serializers.py
│   │   ├── selectors.py
│   │   ├── services.py                 # Configurable compliance scoring & weekly/monthly rollups
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── reports/                        # Phase 8: Structured reports, versioning & traceability
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                   # ReportTemplate, Report, ReportVersion, ReportSection, ReportEvidenceLink
│   │   ├── serializers.py
│   │   ├── selectors.py
│   │   ├── services.py                 # Report drafting, evidence linking, immutable version control
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   └── ... (continued on next page)
```

14.1 Backend Directory Tree — Document Mgmt, AI, Approvals & Audit (Cont.)

14.1 COMPLETE REPOSITORY DIRECTORY TREE — REPORTING, AI, APPROVALS & AUDIT (CONT.)

```
Civora_Backend/
├── apps/ (continued)
│   ├── document_management/          # Phase 8: Unified preview, PDF & DOCX rendering
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                 # GeneratedDocument, ExportJob
│   │   ├── renderers/                # Single consistent rendering pipeline (HTML -> PDF / DOCX)
│   │   │   ├── __init__.py
│   │   │   ├── pdf_renderer.py
│   │   │   └── docx_renderer.py
│   │   ├── services.py
│   │   ├── tasks.py                  # Async document generation Celery tasks
│   │   ├── views.py
│   │   └── urls.py
│   │
│   ├── ai_assistance/                # Phase 9: AI classification, summarization & draft generation
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                 # AIProcessingRun, AISubmissionAnalysis, AIDraftOutput
│   │   ├── prompts/                  # Versioned structured prompts
│   │   │   ├── __init__.py
│   │   │   ├── classification.py
│   │   │   ├── summarization.py
│   │   │   └── report_drafting.py
│   │   ├── schemas.py                # Pydantic schemas for strict LLM output validation
│   │   ├── services.py               # Evidence-linked drafting, missing-info & duplicate detection
│   │   ├── tasks.py                  # Async AI processing jobs with retry/rate-limit budgets
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── approvals/                    # Phase 10: Review, approval, release & governance
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                 # ReportReview, ReviewComment, ApprovalDecision, ReleaseRecord
│   │   ├── serializers.py
│   │   ├── services.py               # Separation-of-duties checks, revision requests, version locking
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   ├── notifications/                # Phase 11: In-app & email notifications, stakeholder delivery
│   │   ├── __init__.py
│   │   ├── apps.py
│   │   ├── models.py                 # Notification, NotificationPreference, NotificationDeliveryLog
│   │   ├── serializers.py
│   │   ├── services.py               # Event routing, permission-aware recipient resolution
│   │   ├── tasks.py                  # Async email & push notification dispatch
│   │   ├── views.py
│   │   ├── urls.py
│   │   ├── migrations/
│   │   └── tests/
│   │
│   └── audit/                        # Phase 2, 3 & 10: Immutable system-wide audit trail
│       ├── __init__.py
│       ├── apps.py
│       ├── models.py                 # AuditLog (actor, tenant, project, action, metadata, timestamp)
│       ├── selectors.py
│       ├── services.py               # Append-only audit recording service
│       ├── views.py                  # Read-only audit log endpoints for Admin & PE governance
│       ├── urls.py
│       ├── migrations/
│       └── tests/
│
└── ... (continued on next page)
```

14.1 Backend Directory Tree — Infrastructure, Common Utilities, Tests & Docker (Cont.)

14.1 COMPLETE REPOSITORY DIRECTORY TREE — INFRASTRUCTURE, COMMON, TESTS & DOCKER (CONT.)

```
Civora_Backend/ (continued)
├── infrastructure/                  # External provider adapters & low-level infrastructure
│   ├── __init__.py
│   ├── storage/                     # S3 / GCS / MinIO private object storage & presigned URLs
│   │   ├── __init__.py
│   │   └── client.py
│   ├── email/                       # Transactional email provider adapter (SES / SendGrid / SMTP)
│   │   ├── __init__.py
│   │   └── client.py
│   ├── ai_provider/                 # LLM client abstraction (Gemini / OpenAI / Anthropic)
│   │   ├── __init__.py
│   │   ├── base.py
│   │   └── client.py
│   └── logging/                     # Structured JSON logging & correlation IDs
│       ├── __init__.py
│       └── formatters.py
│
├── common/                          # Shared cross-cutting utilities & base classes
│   ├── __init__.py
│   ├── models.py                    # Base UUIDModel, TimestampedModel, TenantScopedModel
│   ├── middleware/                  # Tenant context, request ID, audit context middleware
│   ├── exceptions.py                # Custom domain exceptions & unified DRF exception handler
│   ├── pagination.py                # Standardized cursor & page-number pagination
│   ├── responses.py                 # Standardized API response envelope helpers
│   ├── validators.py                # Reusable validation helpers
│   └── management/
│       └── commands/
│           └── seed_dev.py          # Repeatable development database seeding (`manage.py seed_dev`)
│
├── tests/                           # Cross-module integration, E2E & tenant-isolation tests
│   ├── conftest.py                  # Global pytest fixtures (tenants, roles, authenticated clients)
│   ├── factories/                   # Factory Boy model factories for clean test data
│   ├── integration/                 # Cross-domain workflow tests (Submission -> Report -> Approval)
│   └── security/                    # Multi-tenant isolation & RBAC boundary test suite
│
├── scripts/                         # Entrypoint & operational scripts
│   ├── entrypoint.sh                # Container startup script
│   └── start_worker.sh              # Celery worker & beat startup script
│
├── docker/
│   ├── Dockerfile                   # Multi-stage Python production image
│   └── docker-compose.yml           # Local PostgreSQL, Redis, MinIO (S3), API & Celery worker
│
├── .env.example                     # Documented environment variable template (no secrets)
├── manage.py                        # Django CLI entrypoint
├── pyproject.toml                   # Dependencies, Ruff, Mypy, and Pytest configuration
└── README.md                        # Architecture overview, local setup & API documentation links
```

CIVORA CONTROL PLA

14.2 Multi-Repository API Routing, Initialization & Module Design Rules

14.2 MULTI-REPOSITORY API ROUTING & CLIENT ALIGNMENT

Because the backend serves three distinct client repositories, expose a unified, versioned API (`/api/v1/`) with clear route namespaces and role-based guards:

Client Repository	Primary User Roles	Key API Route Scopes Consumed
Civora_Mobile_App	Field Personnel, Project Engineers	`/api/v1/auth/` , `/api/v1/projects/` (assigned), `/api/v1/submissions/` , `/api/v1/evidence/` , `/api/v1/tasks/` , `/api/v1/notifications/`
Civora-PE_and_Stakeholder_Frontend	`project_engineer` , `read_only_stakeholder`	`/api/v1/auth/` , `/api/v1/projects/` , `/api/v1/submissions/` , `/api/v1/tasks/` , `/api/v1/performance/` , `/api/v1/reports/` , `/api/v1/ai/` , `/api/v1/approvals/` , `/api/v1/notifications/`
Civora-Admin_Frontend	`platform_admin` , Organization Admins	`/api/v1/auth/` , `/api/v1/organizations/` , `/api/v1/users/` , `/api/v1/access-control/` , `/api/v1/projects/` , `/api/v1/integrations/` , `/api/v1/audit/`

14.3 HOW TO INITIALIZE THE BACKEND STRUCTURE (STEP-BY-STEP)

When starting implementation in `Civora_Backend` , follow these steps to scaffold the structure cleanly:

1. **Initialize project & dependency manager (`uv` or `poetry`):**

```
mkdir -p config/settings apps infrastructure/{storage,email,ai_provider,logging}
common/management/commands tests/{factories,integration,security} scripts docker
touch apps/__init__.py infrastructure/__init__.py common/__init__.py
```

2. **Create the core Django project inside `config/` :**

```
django-admin startproject config .
```

   Move `config/settings.py` into `config/settings/base.py` and create `env.py` , `development.py` , `staging.py` , `production.py` , and `test.py` .

3. **Create each domain app inside `apps/` according to the phase sequence:**
   - **Phase 1–3 (Foundation, DB & Auth):** `users` , `organizations` , `access_control` , `authentication` , `audit`
   - **Phase 4 (Projects):** `projects`
   - **Phase 5 (Submission Engine):** `submissions` , `evidence`
   - **Phase 6 (External Channels):** `integrations`
   - **Phase 7 (Operations):** `tasks` , `performance`
   - **Phase 8–10 (Reporting, AI & Governance):** `reports` , `document_management` , `ai_assistance` , `approvals`
   - **Phase 11 (Notifications):** `notifications`

   (Register each app in `INSTALLED_APPS` using its full module path, e.g., `"apps.submissions.apps.SubmissionsConfig"`).
4. **Enforce internal app file conventions:**
   - `models.py` — Database schema, constraints, and indexes only (no heavy business orchestration).
   - `services.py` — Write operations, state transitions, permission checks, audit logging, and transaction boundaries (`@transaction.atomic`).
   - `selectors.py` — Read/query operations always scoped by `organization_id` and `project_id` .
   - `serializers.py` — Input validation and output serialization at the HTTP boundary.
   - `views.py` — Thin DRF `APIView` / `ViewSet` classes that validate input, call a service/selector, and return a standardized response.

14.4 MODULE DESIGN RULES

- Keep API views (`views.py`) thin; delegate all business logic to `services.py` and complex queries to `selectors.py` .
- Inherit tenant-owned models from `TenantScopedModel` (`common/models.py`) to guarantee `organization_id` presence.
- Validate external input at system boundaries using DRF Serializers or Pydantic schemas.
- Enforce authorization in server-side permissions (`access_control/permissions.py`) and service guards—never trust client-supplied `organization_id` or role claims.
- Avoid circular module dependencies; cross-domain coordination belongs in application services.
- Keep provider-specific integration code behind adapters (`apps/integrations/adapters/` and `infrastructure/`).

15.0 Delivery Sequence and Milestone Priorities

This is the recommended implementation order, not a fixed calendar estimate. Estimate effort after repository inspection and requirements confirmation.

Milestone	Scope	Acceptance condition
1. Foundation	Database, migrations, authentication, RBAC, project management	Authorized users can create projects and manage access securely.
2. Submission engine	Submissions, sender attribution, evidence storage, audit trail	A field submission can be received, attributed, stored, and retrieved without losing original content.
3. Channel ingestion	WhatsApp proof of concept and supported webhook ingestion	Supported external events are safely mapped into the submission engine.
4. Operations	Tasks, assignments, activity and performance metrics	Project activities can be assigned and measured using documented rules.
5. Reporting	Reports, evidence selection, versions and document exports	Reports can be prepared from traceable project evidence.
6. Governance	Review, revision, approval and release	Authorized humans control report approval and release.
7. Intelligence	AI assistance and notifications	AI supports drafting and analysis without bypassing governance.
8. Production readiness	Security, testing, monitoring, backups and deployment	Core workflows and controls pass production acceptance tests.

FIRST TWO MILESTONES SHOULD RECEIVE PRIORITY

Milestone 1 — Secure project foundation

- Database architecture
- Authentication and session management
- RBAC and tenant isolation
- Organization and project management

Milestone 2 — Submission and evidence engine

- Manual submission API
- Sender attribution
- Evidence upload and storage
- Submission status workflow
- Immutable source record
- Audit trail and retrieval

After these milestones, implement the reporting workflow. Develop WhatsApp ingestion against the same submission service after confirming provider capabilities.

16.0 First Coding-Agent Task (Task 001 — Architecture & DB Foundation)

Task 001 — Backend architecture and database foundation

Give the coding agent this task before allowing it to implement the business modules.

INSTRUCTIONS

1. Inspect the existing `Civora_Backend` repository and identify its actual framework, package versions, scripts, and current implementation.
2. Document the existing architecture and identify reusable components.
3. Do not replace working configurations without a documented technical reason.
4. Establish the Python/Django backend architecture (`config/` , `apps/` , `infrastructure/` , `common/` , `tests/`) and module boundaries.
5. Configure PostgreSQL, Django ORM, and Redis/Celery if they are not already configured.
6. Design the initial multi-tenant database models (`common.models.TenantScopedModel` , `users` , `organizations` , `access_control` , `projects` , `audit`).
7. Define organization, user, membership, role, permission, and project relationships.
8. Configure Django migrations and repeatable development seed management command (`python manage.py seed_dev`).
9. Implement database connection handling, CORS configuration for the three frontend repositories, and environment variable validation.
10. Add initial `pytest` tests for database connectivity and tenant isolation.
11. Document architectural decisions, assumptions, and unresolved requirements.
12. Do not start WhatsApp, AI, or report generation in this task.

REQUIRED OUTPUTS

- Repository architecture assessment
- Standardized Django backend folder structure (`config/` , `apps/` , `infrastructure/` , `common/` , `tests/`)
- Initial Django ORM models (`users` , `organizations` , `access_control` , `projects` , `audit`)
- Initial Django migrations
- Development seed management command (`seed_dev.py`)
- Environment variable template (`.env.example`) without secrets
- Database connection & settings implementation (`config/settings/`)
- Initial `pytest` test suite
- Architecture decision notes
- List of unresolved product and technical questions

ACCEPTANCE CRITERIA

- The application starts successfully.
- PostgreSQL connection works.
- Migrations apply to a clean development database.
- Development seed data can be created repeatably.
- Environment variables are validated.
- Initial database and authorization tests pass.
- No existing working feature has been broken.
- Assumptions and unresolved requirements are documented.

17.0 Non-Negotiable Engineering Principles & Final Recommendation

1. **Evidence integrity:** Preserve original field submissions and external payloads.
2. **Tenant isolation:** Enforce organization and project boundaries server-side.
3. **Human accountability:** AI does not verify facts, approve reports, or authorize release.
4. **Traceability:** Report statements and attachments should be traceable to source records wherever possible.
5. **Explicit authorization:** Role names alone are insufficient; enforce specific permissions and project membership.
6. **Auditability:** Sensitive changes, decisions, and workflow transitions must be attributable.
7. **Idempotency:** External events and retryable operations must not create duplicate records.
8. **Recoverability:** Backups, retries, and failure recovery must be designed rather than added as an afterthought.
9. **Incremental delivery:** Each phase must have testable acceptance criteria.
10. **No fabricated data:** Missing information remains missing or pending verification.

18.0 Final Implementation Recommendation

Treat the **submission and evidence engine** as Civora's central backend capability.

WhatsApp ingestion, AI processing, performance analysis, and report generation should feed into or operate on that engine. This creates a coherent, auditable system instead of a collection of disconnected features.

Start with Task 001. Review the repository and database design before implementing the remaining modules. Confirm uncertain product rules—especially WhatsApp group ingestion, performance scoring, stakeholder visibility, and report approval authority—before encoding them as permanent business logic.

CIVORA
CIVORA CONTROL PLANE · OFFICIAL ENGINEERING ARCHITECTURE ROADMAP

AUTHORIZING PLATFORM ARCHITECT

Civora Engineering Architecture Board

COREN / System Architecture Specification • Stamp #CIV-BE-2026

SUPERVISING TECHNICAL REVIEWER

Approved for Phase 1–12 Execution

Multi-Repository Governance & Security Control Plane